



MQTT & IoT

SEMESTRE DE VERÃO 2025/2026

Agenda

- Introdução & Conceitos Básicos
- Modelo MQTT versus Cliente/Servidor
- MQTT no contexto da IoT
- Exemplos
- Visualização de Dados
- Persistência de Dados
- Ambiente Disponível para Testes

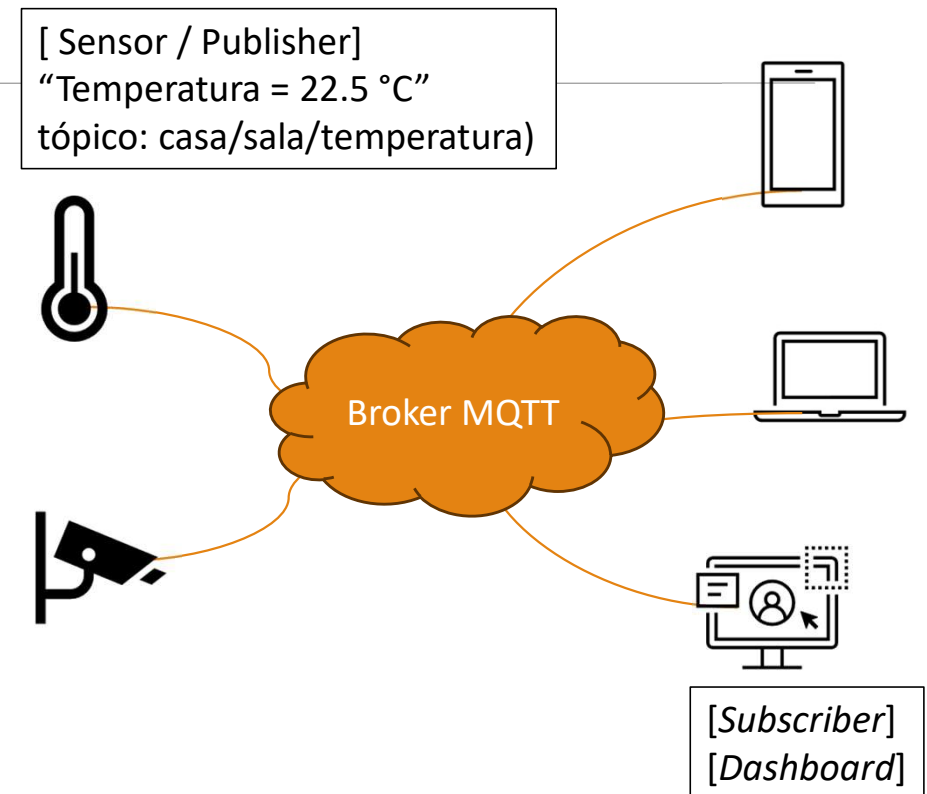


Introdução & Conceitos Básicos

- O crescimento da IoT (*Internet of Things* / Internet das Coisas) exige a utilização de protocolos leves, eficientes e confiáveis
- O protocolo MQTT (*Message Queuing Telemetry Transport*) foi desenvolvido pela IBM (1999) para utilização em redes de largura de banda limitada e assenta sobre:
 - Modelo *publish / subscribe*
 - Comunicação assíncrona utilizando um *broker* como intermediário

Conceitos MQTT

- **Publisher**
 - publica mensagens em tópicos
- **Subscriber**
 - subscreve tópicos e recebe mensagens
- **Broker**
 - servidor central que intermedia toda a comunicação



Conceitos MQTT – *Broker*

- Núcleo central da comunicação
- Responsável por:
 - Receber, num tópico, mensagens dos ***publishers***
 - Distribuir mensagens apenas aos ***subscribers*** do tópico
 - Gerir ligações e sessões
 - Garantir a qualidade do serviço (QoS – *Quality of Service*)
- Exemplos de *brokers*
 - Mosquitto (*open source*)
 - EMQX
 - HiveMQ

Conceitos MQTT – *Broker* – QoS

- A Qualidade de Serviço define o **nível de fiabilidade** com que as mensagens são entregues entre o cliente MQTT e o *broker*.
- Existem três níveis:
 - QoS 0 – *At most once*
 - A mensagem é enviada uma única vez, sem confirmação
 - Pode ser perdida se houver falha na rede
 - Ideal para dados não críticos, como valores de sensores atualizados frequentemente
 - QoS 1 – *At least once*
 - A mensagem é enviada até o *broker* confirmar a receção com um **PUBACK**
 - Pode ser entregue mais de uma vez se houver retransmissão, e.g. alerta de porta aberta
 - QoS 2 – *Exactly once*
 - Utiliza um *handshake* de 4 passos (**PUB**, **PUBREC**, **PUBREL**, **PUBCOMP**)
 - Evita duplicação, com entrega única e garantida.
 - Utiliza mais recursos devendo ser apenas quando é preciso evitar duplicados, e.g. abrir uma válvula crítica

Conceitos MQTT – *Broker* – QoS

Nível	Garantia de Entrega	Duplicação de Mensagens	Complexidade
QoS 0	✗ Não	✗ Não	Baixa
QoS 1	✓ Sim	✓ Sim	Média
QoS 2	✓ Sim	✗ Não	Alta

Conceitos MQTT

- Mensagens Retidas (*Retained Messages*)
 - Quando um cliente publica uma mensagem com a *flag* `retained=true`, o *broker* guarda essa mensagem como a "última mensagem conhecida" para esse tópico.
 - Um novo cliente subscrito ao tópico recebe imediatamente a última mensagem publicada, mesmo que tenha sido publicada antes da subscrição.
- *Keep Alive*
 - O intervalo de tempo máximo (em segundos) que o cliente pode ficar sem comunicar com o *broker*
 - O cliente envia periodicamente mensagens de `PINGREQ`
 - O *broker* responde com `PINGRESP`
 - Se o *broker* não receber nada dentro do tempo limite, considera o cliente desligado

Conceitos MQTT

- *Last Will and Testament* (LWT)
 - Uma mensagem pré-configurada que será enviada pelo *broker* se o cliente desligar-se abruptamente (sem envio de `DISCONNECT`)
 - Definida no momento de conexão do cliente e inclui:
 - Tópico
 - *Payload* (mensagem)
 - QoS
 - *Retain*
 - Útil para:
 - Avisar outros clientes de que um dispositivo está *offline* de forma inesperada
 - Monitorização do estado da rede

Modelo MQTT versus Cliente/Servidor

Característica	Cliente-Servidor	MQTT (<i>Pub/Sub</i>)
Conexão	Direta entre cliente e servidor	Mediada por um <i>broker</i>
Acoplamento	Forte	Fraco
Escalabilidade	Limitada	Alta
Comunicação	Síncrona (Pedido/Resposta)	Assíncrona (Baseada em Eventos)
Robustez a falhas	Reduzida	Alta (QoS)
Exemplo de uso	Aplicações Web	Sistemas IoT, sensores

MQTT no contexto da IoT

- Vantagens em IoT
 - Baixo consumo de largura de banda
 - Permite milhares de dispositivos ligados
 - Fiável em redes instáveis
 - Comunicação eficiente e em tempo real
- Cenários de Aplicações
 - Casas inteligentes (temperatura, iluminação, segurança)
 - Agricultura inteligente (chuva, solo, humidade, temperatura)
 - Indústria 4.0
 - Saúde remota

Exemplos Práticos

Publicação de dados com ESP32

```
#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid = "REDE_WIFI";
const char* password = "PASSWORD";
const char* mqtt_server = "MQTT_SERVER";
const char* mqtt_user = "MQTT_USER";
const char* mqtt_password = "MQTT_PASSWORD";
const char* mqtt_topic = "/casa/sala/temperatura";

WiFiClient espClient;
PubSubClient client( espClient );

void setup_wifi() {
  WiFi.begin( ssid, password );
  while ( WiFi.status() != WL_CONNECTED) delay( 500 );
}

void setup() {
  Serial.begin( 115200 );
  setup_wifi();
  client.setServer( mqtt_server, 1883 );
}
```

```
void loop() {
  if ( !client.connected() )
    client.connect( "ESP32Client", mqtt_user, password);

  float temp = 22.5 + random(-10, 10)/10.0;

  char msg[10];

  dtostrf(temp, 4, 2, msg);
  client.publish( mqtt_topic, msg );

  delay(10000);
}
```

Subscrição de dados com Python

```
import paho.mqtt.client as mqtt

MQTT_SERVER = "MQTT_SERVER"
MQTT_PORT = 1883
MQTT_TOPIC = "/casa/sala/temperatura"
MQTT_USER = "MQTT_USER"
MQTT_PASSWORD = "MQTT_PASSWORD"

def on_connect(client, userdata, flags, reasonCode, properties=None):
    print("Ligado ao broker com código: " + str(reasonCode))
    if reasonCode != mqtt.CONNACK_ACCEPTED:
        print("Erro na ligação: não autorizado.")

def on_message(client, userdata, msg):
    print( f"{msg.topic} -> {msg.payload.decode}" )
```

```
client = mqtt.Client()

client.username_pw_set( MQTT_USER, MQTT_PASSWORD )

client.on_connect = on_connect
client.on_message = on_message

client.connect( MQTT_SERVER, MQTT_PORT, keepalive=60 )

client.loop_forever()
```

Visualização de dados

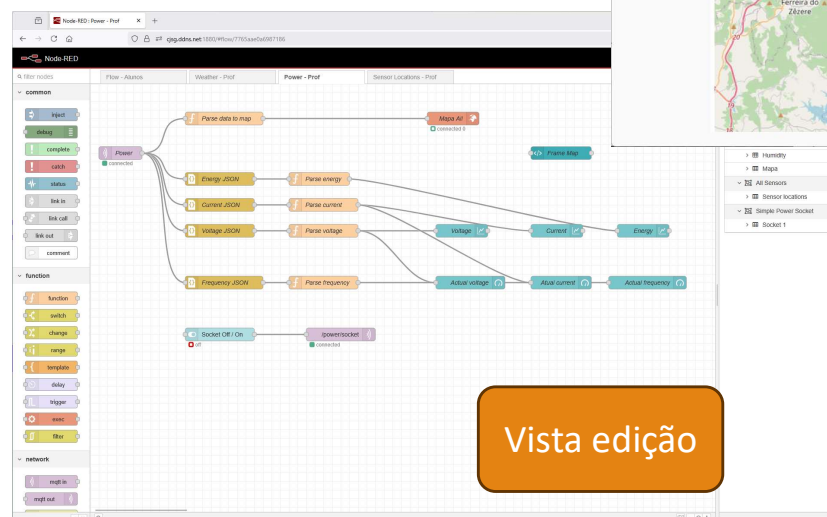
• Node-RED



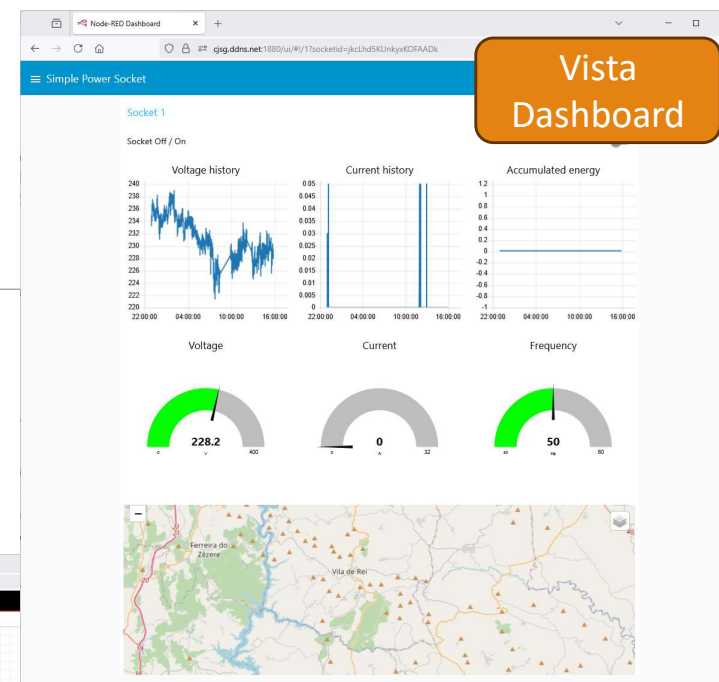
• Plataforma visual de programação

• Permite:

- Subscrição de tópicos MQTT
- Transformações em tempo real
- Visualização com gráficos, *gauges*, etc.



Vista edição



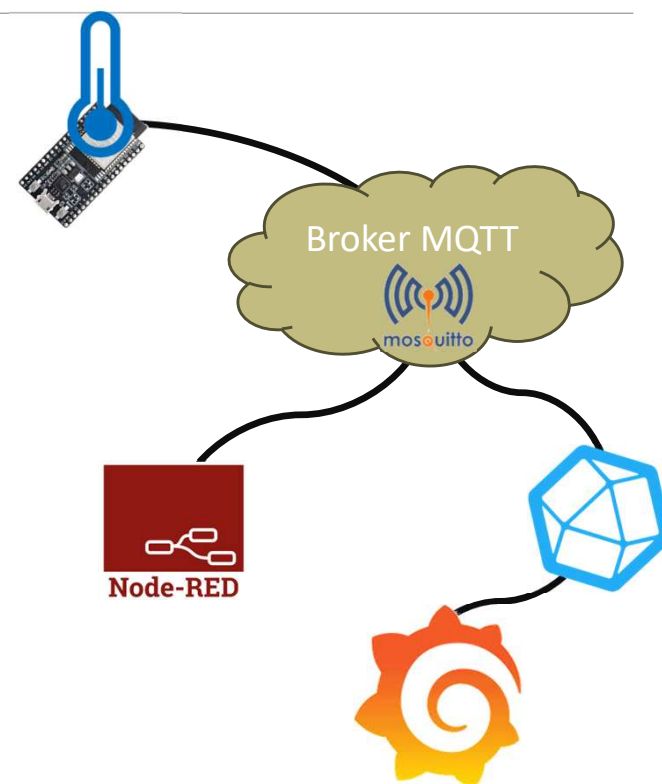
Vista
Dashboard

Persistência dos Dados

- Por que guardar os dados?
 - Análise posterior
 - Geração de alertas
 - *Dashboards* históricos
- Alimentar sistemas de IA (Inteligência Artificial) ou *Machine Learning* (ML)
- Formas comuns de persistência:
 - Ficheiros CSV ou JSON
 - Bases de dados relacionais (MySQL, PostgreSQL)
 - Bases de dados temporais como InfluxDB

Persistência dos Dados – InfluxDB

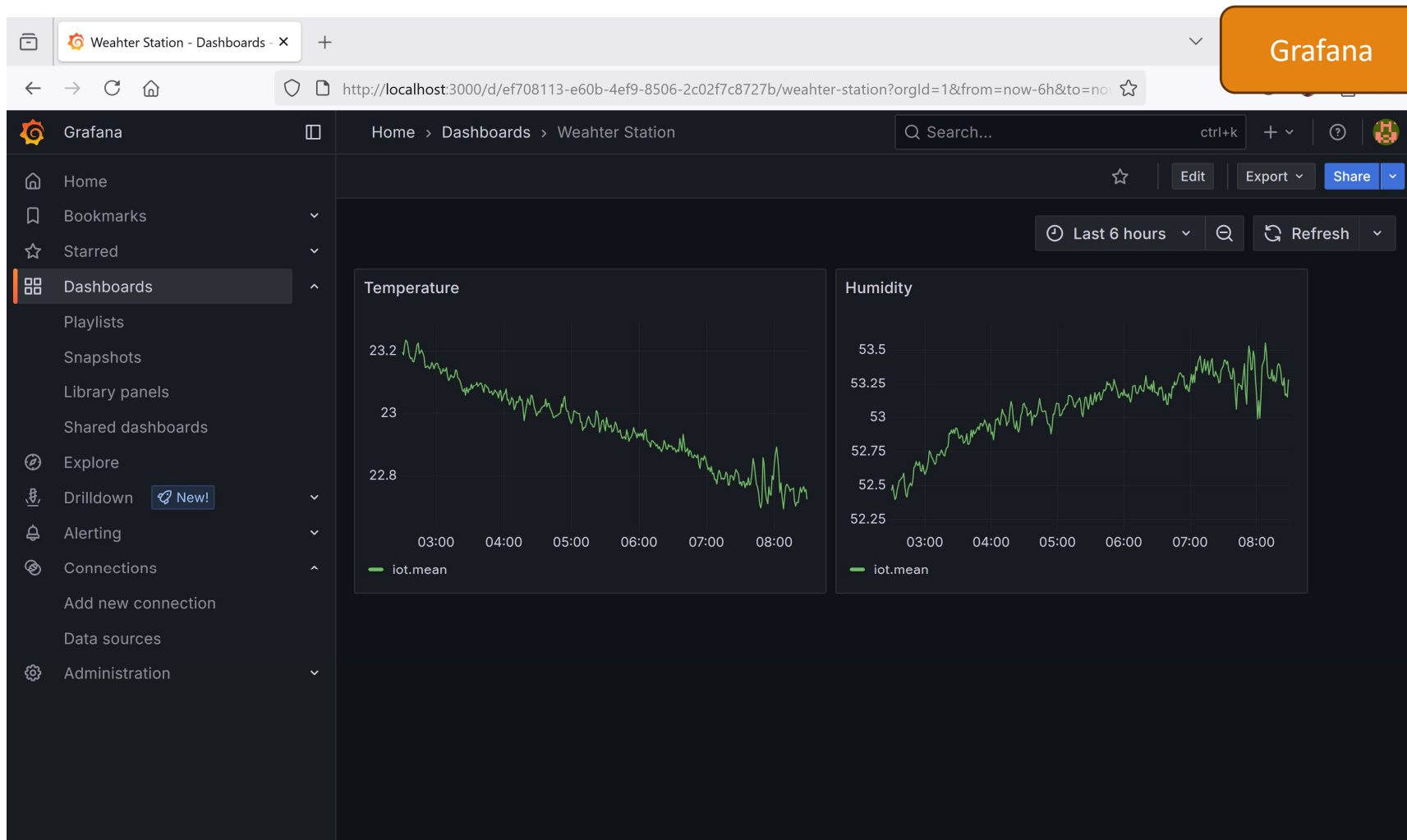
- Características dos dados:
 - Dados com *timestamp*
 - Frequência alta e contínua
 - Importância das consultas temporais
- Vantagens de utilização do InfluxDB
 - Otimizado para séries temporais
 - Escrita rápida e leve
 - Retenção de dados automática
 - Consultas temporais simples
 - Integração fácil com Grafana (para visualização de dados)



Visualização dos Dados – Grafana

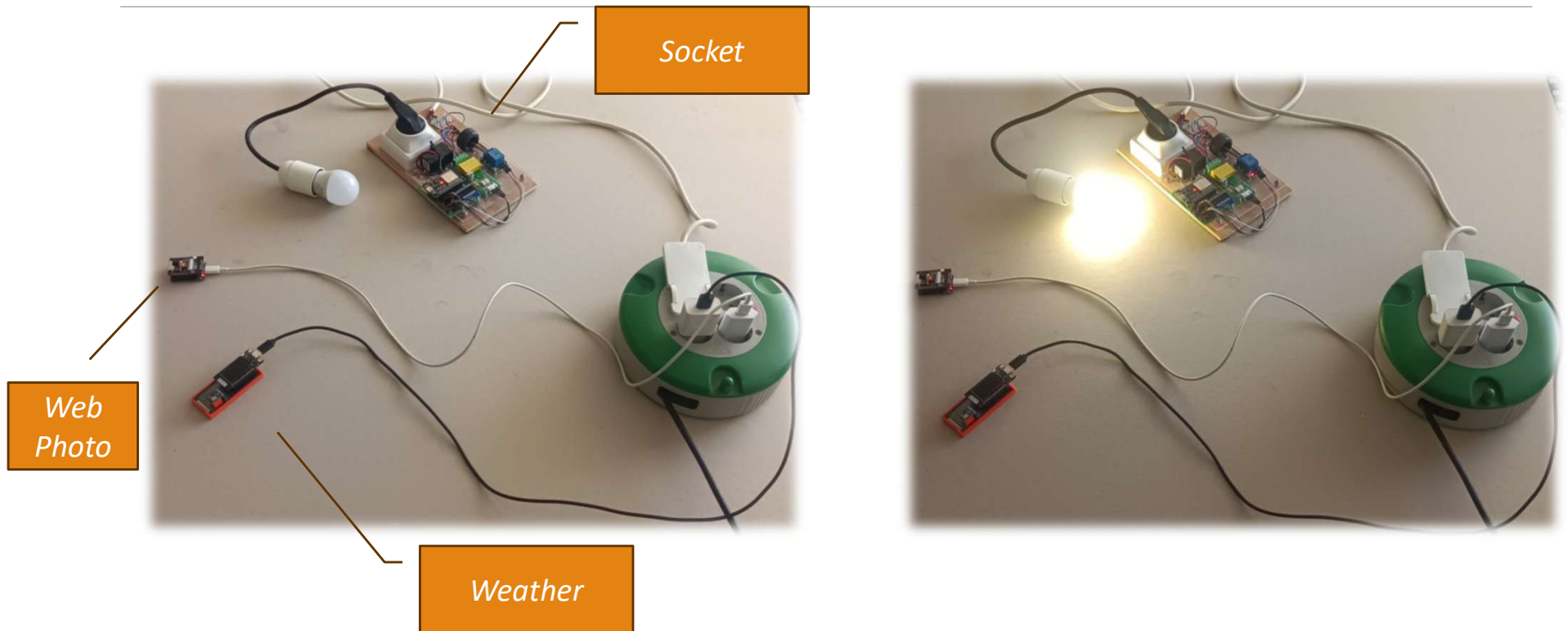


- O que é o Grafana
 - Plataforma *open source* de monitorização e visualização
 - Suporta múltiplas bases de dados (InfluxDB, Prometheus, etc.)
 - Interface web rica e interativa
 - Permite criar *dashboards* com:
 - Gráficos de séries temporais
 - Medidores (*gauges*)
 - Tabelas, alertas etc.

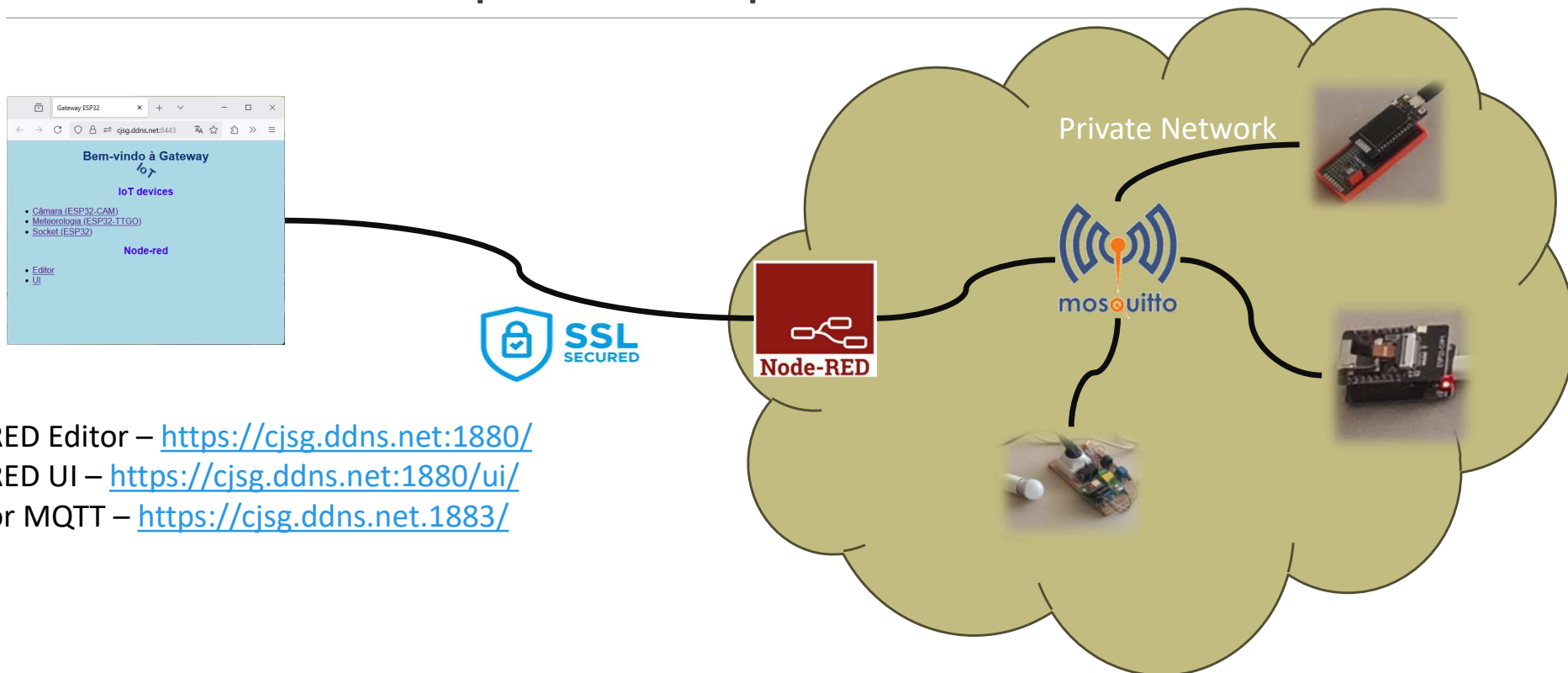




Ambiente Disponível para Testes



Ambiente Disponível para Testes – MQTT

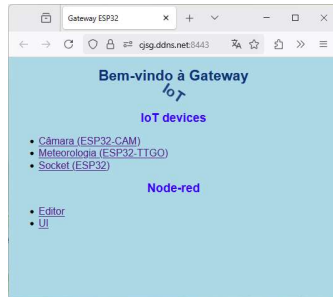


Node-RED Editor – <https://cjsq.ddns.net:1880/>

Node-RED UI – <https://cjsq.ddns.net:1880/ui/>

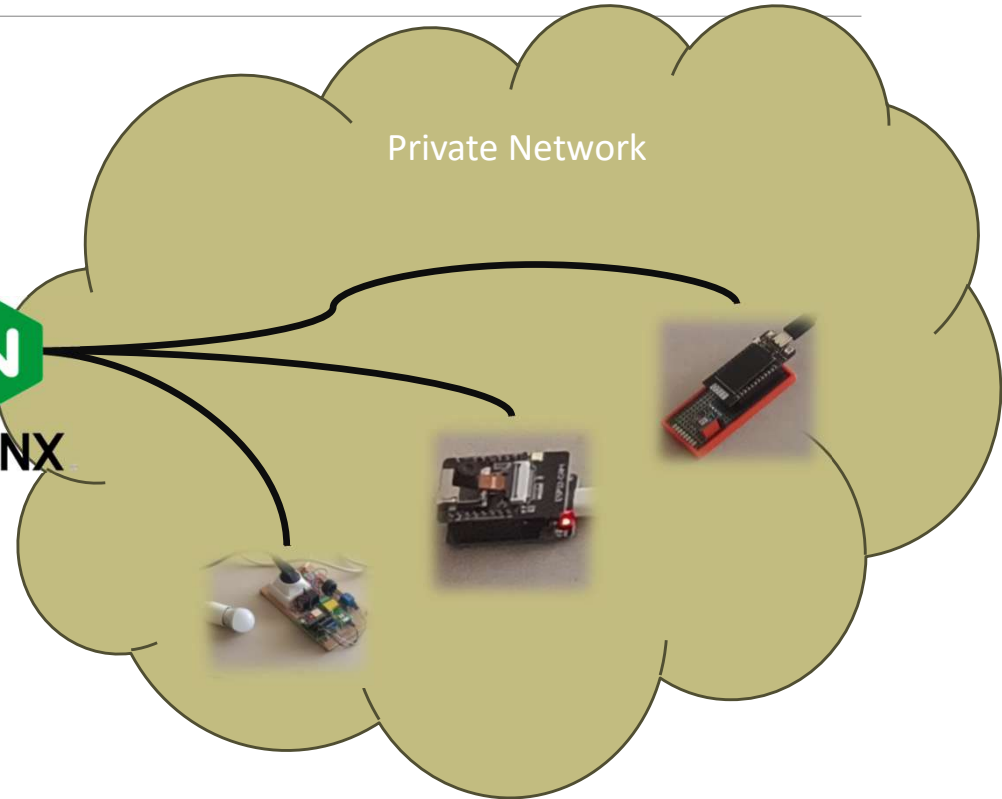
Servidor MQTT – <https://cjsq.ddns.net.1883/>

Ambiente Disponível para Testes – Direto

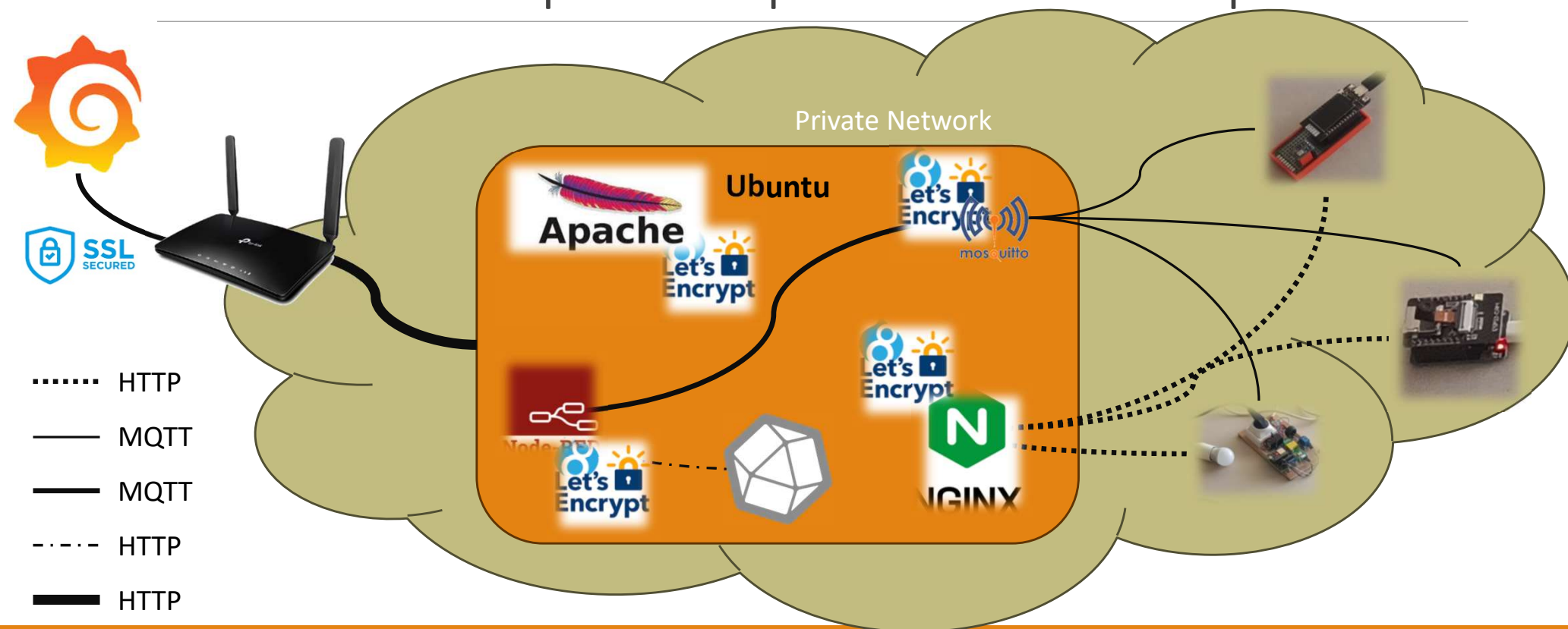


Private Network

ESP Gateway – <https://cjsd.ddns.net:8443/>
Web Photo 1 – <https://cjsd.ddns.net:8443/stream1/>
Web Photo 2 – <https://cjsd.ddns.net:8443/stream2/>
Weather – <https://cjsd.ddns.net:8443/weather/>
Socket – <https://cjsd.ddns.net:8443/socket/>



Ambiente Disponível para Testes - Arquitetura



Ambiente Disponível para Testes - Arquitetura

- Serviços disponíveis diretamente através de *port forward* no router:
 - Apache HTTP (porto 443)
 - Node-RED (porto 1880)
 - Mosquitto (port 1883)
- Serviços disponíveis através de *proxy* com Nginx (porto 8443):
 - Dispositivos IoT
 - Servidor InfluxDB
- O certificado SSL é comum a todos os serviços (Let's Encrypt – <https://letsencrypt.org/>)

Ambiente Disponível para Testes

Comandos REST

- Web Photo
 - Direto (GET → image)
 - /stream1/jpg
 - MQTT
 - Topic name
 - /stream1
 - Values (JSON)
 - name
 - latitude
 - longitude
 - endPoint
- Weather
 - Direto (GET → JSON)
 - /weather/values
 - /weather/position
 - MQTT
 - Topic name
 - /weather
 - Values (JSON)
 - temperature
 - humidity
 - time
 - name
 - latitude
 - longitude
- Socket
 - Direto (GET → JSON)
 - /socket/values
 - /socket/position
 - /socket/on
 - /socket/off
 - MQTT
 - Topic name
 - /power
 - Values (JSON)
 - voltage
 - current
 - power
 - energy
 - frequency
 - powerFactor
 - time
 - latitude
 - longitude

Desafio

Desenvolva um *dashboard* (Python + JS) que aceda à API REST e apresente o estado atual dos sensores (`Weather` e `Socket`)

A página de *dashboard* deverá ser atualizada a cada 10 segundos

Opcionalmente o *dashboard* deverá incluir um mapa, com um *pin* para cada sensor. Associado a cada *pin* deve existir um menu *pop-up* que permite mostrar os dados do sensor.